TP : Gestion d'un personnel universitaire

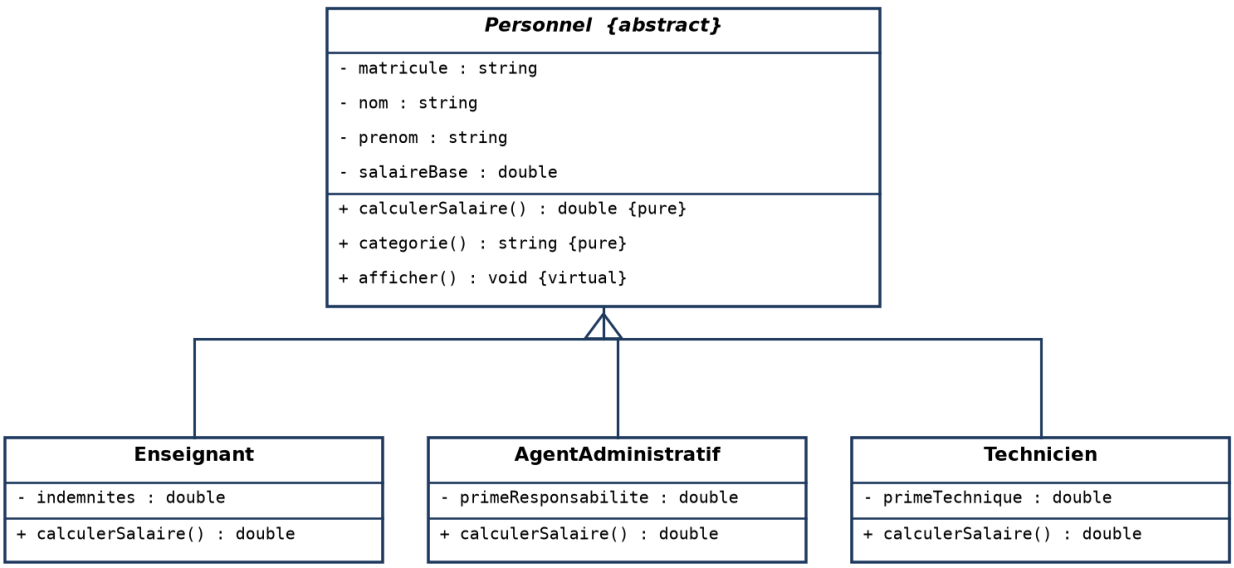
Thèmes : héritage, polymorphisme, classes abstraites, méthodes virtuelles, redéfinition, liaison dynamique.

1. Analyse des classes

Classe mère : `Personnel`. Elle regroupe ce que tous les employés ont en commun : matricule, nom, prénom et salaire de base. Elle est **abstraite** car il n'existe pas d'« employé générique » : on ne crée que des enseignants, agents ou techniciens. Le calcul du salaire dépend de la catégorie, donc il est déclaré *virtuel pur* (`= 0`) dans la mère.

Classe dérivée	Attribut spécifique	Calcul du salaire
Enseignant	indemnites	salaire de base + indemnités
AgentAdministratif	primeResponsabilite	salaire de base + prime de responsabilité
Technicien	primeTechnique	salaire de base + prime technique

2. Diagramme de classes UML



Héritage (généralisation) : chaque classe dérivée hérite de Personnel (flèche à triangle creux vers la classe mère)

Relations : les trois classes dérivées héritent de `Personnel` (relation de généralisation / héritage). `calculerSalaire()` est redéfinie dans chaque dérivée. Les attributs de la mère sont `protected` pour être accessibles aux dérivées.

3. Implémentation en C++

```
#include <iostream>
#include <iomanip>
#include <memory>
#include <string>
#include <vector>
using namespace std;

// ===== Classe mère abstraite =====
class Personnel {
protected:
    string matricule;
    string nom;
    string prenom;
    double salaireBase;

    // Hook virtuel : chaque classe dérivée ajoute ses propres infos
    virtual void afficherDetails() const {}

public:
    Personnel(const string& mat, const string& n, const string& p, double sb)
        : matricule(mat), nom(n), prenom(p), salaireBase(sb) {}

    virtual ~Personnel() = default;

    // Méthodes virtuelles pures : rendent la classe abstraite
    virtual double calculerSalaire() const = 0;
    virtual string categorie() const = 0;

    // Méthode virtuelle : affichage commun à tous les employés
    virtual void afficher() const {
        cout << fixed << setprecision(2);
        cout << "Matricule       : " << matricule << "\n"
             << "Nom             : " << nom << "\n"
             << "Prenom          : " << prenom << "\n"
             << "Categorie       : " << categorie() << "\n"
             << "Salaire de base : " << salaireBase << " EUR\n";
        afficherDetails();
        cout << "Salaire total   : " << calculerSalaire() << " EUR\n";
    }
};

// ===== Classes dérivées =====
class Enseignant : public Personnel {
    double indemnites;
protected:
    void afficherDetails() const override {
        cout << "Indemnites      : " << fixed << setprecision(2) << indemnites << " EUR\n";
    }
public:
    Enseignant(const string& mat, const string& n, const string& p, double sb, double ind)
        : Personnel(mat, n, p, sb), indemnites(ind) {}

    double calculerSalaire() const override { return salaireBase + indemnites; }
    string categorie() const override { return "Enseignant"; }
};

class AgentAdministratif : public Personnel {
    double primeResponsabilite;
protected:
    void afficherDetails() const override {
        cout << "Prime resp.     : " << fixed << setprecision(2) << primeResponsabilite << "
```

```
EUR\n";
    }
public:
    AgentAdministratif(const string& mat, const string& n, const string& p, double sb, double
prime)
        : Personnel(mat, n, p, sb), primeResponsabilite(prime) {}

    double calculerSalaire() const override { return salaireBase + primeResponsabilite; }
    string categorie() const override { return "Agent administratif"; }
};

class Technicien : public Personnel {
    double primeTechnique;
protected:
    void afficherDetails() const override {
        cout << "Prime technique : " << fixed << setprecision(2) << primeTechnique << " EUR\n";
    }
public:
    Technicien(const string& mat, const string& n, const string& p, double sb, double prime)
        : Personnel(mat, n, p, sb), primeTechnique(prime) {}

    double calculerSalaire() const override { return salaireBase + primeTechnique; }
    string categorie() const override { return "Technicien"; }
};

// ===== Programme principal =====
int main() {
    // Collection polymorphe : pointeurs sur la classe mère
    vector<unique_ptr<Personnel>> personnel;
    personnel.push_back(make_unique<Enseignant>("E001", "Dupont", "Marie", 2800.0, 450.0));
    personnel.push_back(make_unique<Enseignant>("E002", "Martin", "Paul", 3100.0, 600.0));
    personnel.push_back(make_unique<AgentAdministratif>("A001", "Bernard", "Sophie", 1900.0,
250.0));
    personnel.push_back(make_unique<Technicien>("T001", "Petit", "Luc", 1750.0, 300.0));
    personnel.push_back(make_unique<Technicien>("T002", "Durand", "Alice", 1800.0, 180.0));

    cout << "===== Fiches du personnel =====\n\n";
    double masse = 0.0;
    for (const auto& p : personnel) {
        p->afficher();            // liaison dynamique : la bonne version est appelée
        cout << "\n";
        masse += p->calculerSalaire();
    }

    cout << fixed << setprecision(2);
    cout << "Masse salariale totale : " << masse << " EUR\n";
    return 0;
}
```

4. Justification des notions utilisées

Notion	Où dans le code
Héritage	`class Enseignant : public Personnel` (idem pour les deux autres)
Classe abstraite	`Personnel` possède des méthodes virtuelles pures (`= 0`), donc on ne peut pas l'instancier directement
Méthodes virtuelles	`calculerSalaire()`, `categorie()`, `afficher()`, `afficherDetails()`
Redéfinition	Mot-clé `override` dans chaque classe dérivée
Polymorphisme et liaison dynamique	`vector<unique_ptr<Personnel>>` : la boucle `p->afficher()` et `p->calculerSalaire()` appelle automatiquement la version de la vraie classe au moment de l'exécution

Destructeur virtuel	`virtual ~Personnel()` : libère correctement les objets dérivés via un pointeur sur la mère